

01 / 졸업작품 · 3인 팀 · 2025.03-06

AI 요리 어시스턴트 장금이

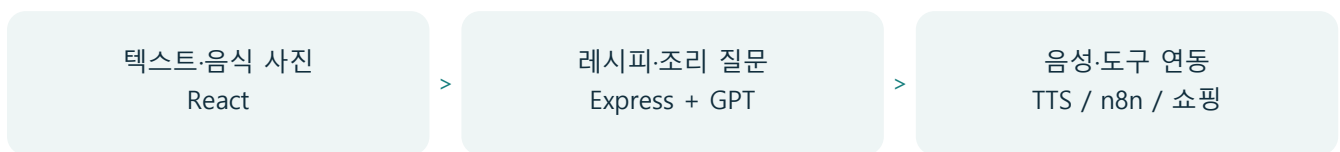
텍스트·음식 사진에서 레시피를 찾고, 조리 중 질문과 음성 안내로 이어지는 웹 서비스

문제

요리 중에는 레시피를 다시 읽는 것보다 현재 상황을 설명하고 바로 답을 받는 흐름이 필요했습니다. 레시피 탐색, 조리 단계 안내, 재료 검색을 하나의 화면 흐름으로 연결했습니다.

구현

React와 Express를 중심으로 GPT-4o의 텍스트·이미지 입력을 연결했습니다. 요리명·재료·조리 단계는 JSON Schema로 구조화하고, 조리 질문의 응답은 단계 이동과 타이머 같은 화면 동작으로 이어지도록 구성했습니다.



기술 선택

초기 LLaMA3-8B 파인튜닝 실험에서는 상용 API 대비 뚜렷한 품질 이점을 얻지 못하고 응답 지연도 확인했습니다. 최종적으로 자체 모델 학습보다 상용 LLM과 외부 도구를 연결하는 구현에 집중했습니다.

코드로 확인할 수 있는 내용

레시피 생성·조리 보조·TTS·쇼핑 API·n8n Webhook 호출을 남겼습니다. 자막 커스텀 노드는 기존 자막 라이브러리를 n8n에서 호출하도록 감싼 구현이며, 전체 n8n 워크플로우와는 구분합니다.

범위와 한계

팀의 구현 결과를 기준으로 소개합니다. 외부 서비스까지 연결한 현재의 전체 실행 검증은 수행하지 않았습니다. 음성 응답 지연과 모바일 최적화는 당시 개선 과제였으며, 미완성 인증 흐름은 핵심 성과에서 제외했습니다.

[README · 선별 코드 보기](#)

02 / 개인 프로젝트 · 머신러닝 · 2024.2학기

레시피 맛 척도 예측

재료 종류와 양으로 매운맛·단맛·짭맛·감칠맛을 추정하는 4출력 회귀 모델

문제

레시피의 재료 비율과 맛 사이의 관계를 모델로 표현하고자 했습니다. 다만 맛을 직접 평가한 데이터가 부족해, 입력을 정리하는 일과 학습에 사용할 정답을 정의하는 일이 먼저 필요했습니다.

데이터와 정답 설계

보고서상 닭볶음탕 레시피 705건을 수집하고, 유사한 재료 이름과 서로 다른 계량 단위를 정리했습니다. 댓글이 부족한 문제에는 재료별 맛 기여 규칙과 댓글 분석을 결합하는 대리 정답 설계로 접근했습니다.

재료명 통일
단위 → g

>

재료별 양
벡터화

>

Dense 회귀 모델
네 가지 맛 점수

모델과 실험

Dense 256→128→64→4 구조에 Dropout을 적용하고 Adam과 Huber loss로 학습했습니다. 손실함수와 모델 크기 등을 비교했으며, 회귀 지표인 MAE·RMSE로 오차를 확인하는 코드를 작성했습니다.

이번 정리에서 확인한 것

현재 최종 재료 파일과 점수표가 파일명으로 연결되는 데이터는 449건입니다. 실행본은 원본의 순서 의존성을 줄이기 위해 파일명으로 입력과 정답을 결합하고, 재료 사전을 저장하도록 보완했습니다.

해석과 한계

대리 정답을 예측하는 오차는 실제 사람이 느끼는 맛의 정확도가 아닙니다. 조리 순서·시간·온도와 실제 관능 평가가 빠져 있으며, 과거 보고서의 검증 손실을 이번에 재현한 성능처럼 제시하지 않습니다.

[README](#) · [원본 복원본](#) · [실행본 보기](#)

03 / 개인 프로젝트 · 데이터와 패턴분류 · 2025.1학기

와인 품질 분류

성분 변수 선택과 신경망 구조 비교를 통해 분류 모델의 학습 과정을 분석한 프로젝트

문제

1,599개 와인 샘플의 화학 성분으로 품질 등급을 분류했습니다. 품질 등급이 특정 구간에 몰려 있는 상황에서, 입력 변수와 모델 복잡도가 학습에 미치는 영향을 살펴보았습니다.

변수 선택

11개 성분의 상관계수와 RandomForest 중요도를 함께 비교했습니다. 두 분석에서 상대적으로 기여가 낮다고 판단한 잔류당과 유리 이산화황을 제외해 9개 특성을 입력으로 구성했습니다.

성분 11개
상관·중요도 비교

>

입력 9개
스케일링

>

신경망 분류
품질 등급

모델 비교

은닉층 크기와 Dropout 비율을 바꾸며 훈련·검증 곡선을 비교했습니다. 남아 있는 제출 코드에서는 Dense 64→32 구조, BatchNormalization, Dropout 0.4·0.3, SGD를 확인할 수 있습니다.

기록과 코드의 차이

보고서에는 128→64 구조와 약 65.8%의 결과가 기술돼 있으나 제출 코드와 다릅니다. 현재 코드의 재현 성능으로 해당 수치를 사용하지 않고, 실험 당시의 관찰과 보존된 구현을 구분했습니다.

검증 설계에 대한 후속 검토

원본은 전체 데이터로 변수 중요도와 스케일을 확인한 뒤 분할합니다. 재평가 시에는 먼저 데이터를 나누고 훈련 데이터에서만 전처리를 학습해야 합니다. 독립적인 테스트 데이터도 분리해 확인할 필요가 있습니다.

[README · 당시 제출 코드 보기](#)